Translating Term Rewrite Systems to Directed Complexes

Contents

Contents 1
1 Usage 1
2 Example 1
3 The polygraphic encoding 2
3.1 Background: polygraphs 3
3.2 Dimension 0: the point 3
3.3 Dimension 1: the sort 3
3.4 Dimension 2: function symbols and structural cells 3
3.5 Encoding a term as a 2-morphism 4
3.6 Handling constants 5
3.7 Dimension 3: rewrite rules and structural equations 5
3.8 Summary of the encoding 5
References 6

1 Usage

The converter lives in `trs/` and is written in TypeScript. Build and run as follows.

Build.

```
cd trs/
npx tsc          # compiles to dist/
```

Run.

```
node dist/main.js <input.ari> [output.ali]
```

If no output path is given, the tool writes to the input path with extension changed to `.ali`. Use `-` as the output to print to stdout.

Verify. Feed the output to alifib:

```
alifib output.ali
```

The tool reads the ARI format used by TPDB: S-expression declarations `(format TRS)`, `(fun` f n`)`, and `(rule` ℓ r`)`. Quoted symbols `|0|` are unquoted and sanitized into valid alifib identifiers.

2 Example

Consider the following TRS, expressing that a unary operation minus is an involutive anti-homomorphism with respect to a unary h and a binary f.

Input: `involution.ari`

```
(format TRS)
(fun minus 1)
(fun h 1)
(fun f 2)
(rule (minus (minus x)) x)
```


Author's Contact Information:

```
(rule (minus (h x)) (h (minus x)))
(rule (minus (f x y)) (f (minus y) (minus x)))
```

Running the converter produces the following alifib file.

Output: `involution.ali`

```
@Type
Eq ≪= {
  pt,
  dom : pt → pt, cod : pt → pt,
  lhs : dom → cod, rhs : dom → cod,
  dir : lhs → rhs, inv : rhs → lhs
},

involution ≪= {
  pt,
  ob : pt → pt,

  (* Structural cells *)
  copy : ob → ob ob,
  swap : ob ob → ob ob,

  (* Function symbols *)
  minus : ob → ob,
  h     : ob → ob,
  f     : ob ob → ob,

  (* Identity 2-cells *)
  id_1 : ob → ob,
  id_2 : ob ob → ob ob,

  (* Naturality of copy *)
  attach Copy_minus :: Eq along
    [ lhs => minus copy,
      rhs => copy (minus ⊗ minus) ],
  attach Copy_h :: Eq along
    [ lhs => h copy,
      rhs => copy (h ⊗ h) ],
  attach Copy_f :: Eq along
    [ lhs => f copy,
      rhs => (copy ⊗ copy) (ob swap ob) (f ⊗ f) ],

  (* Structural equations *)
  id_1_idem     : id_1 id_1 → id_1,
  id_1_idem_inv : id_1 → id_1 id_1,
  swap_inv      : swap swap → id_2,
  swap_inv_inv  : id_2 → swap swap,
  copy_comm     : copy swap → copy,
  copy_comm_inv : copy → copy swap,

  (* Rewrite rules *)
  rule_1 : minus minus → id_1,
```

```
  rule_2 : h minus → minus h,
  rule_3 : f minus → swap (minus ⊗ minus) f
}
```

The three rewrite rules translate to 3-cells whose source and target are the two sides of each rule, encoded as 2-morphisms in the polygraph.

3 The polygraphic encoding

The translation from a TRS to a directed complex follows the standard theory of polygraphic rewriting [1, 2]. We briefly recall the key ideas and then describe the details of the encoding.

3.1 Background: polygraphs

A *polygraph* (also called a *computad*) is a higher-dimensional presentation of an algebraic structure. Just as a group presentation has generators and relations, a polygraph has generators at each dimension:

- 0-**cells** are points (objects).
- 1-**cells** are arrows between points.
- 2-**cells** are rewrites between (composites of) arrows.
- 3-**cells** are rewrites between 2-cell composites.

In the context of term rewriting, a TRS with signature Σ and rules R gives rise to a $(3, 2)$-polygraph: function symbols are 2-generators and rewrite rules are 3-generators.

The key insight is that terms are *tree-shaped* (they can share and discard variables), while cells in a polygraph are *string-diagram-shaped* (every input wire goes to exactly one output). To bridge this gap, we introduce *structural cells* that mediate between the linear world of string diagrams and the non-linear world of terms.

3.2 Dimension 0: the point

Every complex begins with a single 0-cell:

$$\mathsf{pt}$$

This is the unique object of the underlying monoid (or, if one prefers, the unique 0-cell of the one-object free higher category).

3.3 Dimension 1: the sort

For an unsorted (single-sorted) TRS, we introduce one 1-cell:

$$\mathsf{ob} : \mathsf{pt} \to \mathsf{pt}$$

This plays the role of the unique sort. A term of the TRS with n free variables will be encoded as a 2-morphism $\mathsf{ob}^n \to \mathsf{ob}$, where ob^n denotes the n-fold horizontal composite $\mathsf{ob} \otimes \cdots \otimes \mathsf{ob}$.

When the TRS has *constants* (function symbols of arity 0) or *erasing rules* (rules where a variable appears on the left but not on the right), we also need a second 1-cell:

$$\mathsf{unit} : \mathsf{pt} \to \mathsf{pt}$$

This is the monoidal unit stand-in. A constant c is then a 2-cell $\mathsf{unit} \to \mathsf{ob}$ rather than $\to \mathsf{ob}$ (which is not syntactically well-formed as a boundary in alifib).

3.4 Dimension 2: function symbols and structural cells

3.4.1 Function symbols. Each function symbol f of arity n becomes a 2-cell:

$$f : \underbrace{\mathsf{ob}\ \mathsf{ob}\ \cdots\ \mathsf{ob}}_{n} \to \mathsf{ob}$$

For a constant c of arity 0:

$$c : \mathsf{unit} \to \mathsf{ob}$$

A term $f(t_1, \ldots, t_n)$ is then encoded by first encoding each t_i as a 2-morphism (recursively), composing them horizontally with $\otimes$, and then composing vertically with f. For example, $h(\mathsf{minus}(x))$ is encoded as $\mathsf{minus} \cdot h$ (apply minus, then h), and $f(t_1, t_2)$ is encoded as $(t_1 \otimes t_2) \cdot f$ where $\otimes$ is horizontal composition (`#0` in alifib).

3.4.2 Structural cells. String diagrams are *linear*: each input wire is used exactly once. Terms are not. To handle variable sharing (contraction) and discarding (weakening), we introduce:

$: \mathsf{ob} \to \mathsf{ob}\ \mathsf{ob}$	(duplicate a wire)
$\mathsf{swap} : \mathsf{ob}\ \mathsf{ob} \to \mathsf{ob}\ \mathsf{ob}$	(exchange two wires)
$\mathsf{erase} : \mathsf{ob} \to \mathsf{unit}$	(discard a wire)

The erase cell is only generated when the TRS has constants or erasing rules. Together with the monoidal unit eliminators $\mathsf{unit_l} : \mathsf{unit}\ \mathsf{ob} \to \mathsf{ob}$ and $\mathsf{unit_r} : \mathsf{ob}\ \mathsf{unit} \to \mathsf{ob}$, these cells allow us to convert any term into a string diagram over the signature.

3.4.3 Identity cells. In alifib, the horizontal composite $\mathrm{id}_{\mathrm{ob}} \otimes \mathrm{id}_{\mathrm{ob}}$ (written `id #0 id`) produces a non-globular diagram that cannot serve as the boundary of a higher cell. We therefore introduce explicit identity 2-cells:

$$\mathrm{id}_n : \mathrm{ob}^n \to \mathrm{ob}^n \qquad \text{for each } n \text{ up to the maximum arity}$$

In practice, id_1 and id_2 are always generated, and higher id_n are added as needed.

3.5 Encoding a term as a 2-morphism

Given a rule $\ell \to r$ with variables $x_1, \ldots, x_k$ (those appearing in ℓ, in left-to-right first-occurrence order), each side is encoded as a 2-morphism $\mathrm{ob}^k \to \mathrm{ob}$. The encoding of a term t starting from input wires $x_1, \ldots, x_k$ proceeds in two phases.

Phase 1: Gather. The *gather phase* routes the k input wires to match the leaf variable pattern of t. This involves three sub-steps:

(1) **Copy:** For each variable x_i that appears m times in t, apply a copy tree $\mathrm{ob} \to \mathrm{ob}^m$. For a variable that appears 0 times, apply erase. These operations are composed horizontally across all input wires.
(2) **Eliminate unit wires:** If any variables were erased, the resulting unit wires are removed using unit_l and unit_r.
(3) **Permute:** The copied wires may not be in the order required by t. A *swap network* of adjacent transpositions (using the swap cell) permutes them into the correct leaf order. The algorithm computes a bubble-sort sequence from the current wire arrangement to the target arrangement.

Phase 2: Apply. The *apply phase* recursively encodes the term structure. Each subterm $f(t_1, \ldots, t_n)$ is encoded as:

$$(t_1 \otimes \cdots \otimes t_n) \cdot f$$

where each t_i is encoded recursively. A variable leaf becomes the identity id_1, and a constant leaf becomes the constant cell directly.

Example. Consider the rule $f(x, y) \to f(y, x)$ (commutativity). Both sides have variables $[x, y]$.

- *LHS:* The leaf order of $f(x, y)$ is $[x, y]$, which matches the input order. The gather phase is trivial. The apply phase gives f. So the LHS encoding is f.
- *RHS:* The leaf order of $f(y, x)$ is $[y, x]$. The gather phase produces swap (one adjacent transposition). The apply phase gives f. So the RHS encoding is $\mathsf{swap} \cdot f$.

The rewrite rule becomes the 3-cell $\mathsf{rule} : f \to \mathsf{swap} \cdot f$.

Example (non-linear). Consider $f(x, x) \to x$ (idempotence of f). Variables: $[x]$.

- *LHS:* The leaf pattern is $[x, x]$. The gather phase produces 3.5. The apply phase gives f. So the LHS encoding is f.
- *RHS:* The leaf pattern is $[x]$. The gather phase is trivial. The apply phase gives id_1. So the RHS encoding is id_1.

The rewrite rule becomes $\mathsf{rule} : f \to \mathrm{id}_1$.

3.6 Handling constants

When a term contains a constant c (arity 0), the encoding needs to produce a unit wire and then apply $c : \mathsf{unit} \to \mathsf{ob}$. Since the input context consists entirely of ob wires, the strategy is:

(1) Make extra copies of a "donor" variable (the first input variable) to create enough ob wires for both variable slots and constant slots.
(2) Permute all wires (while they are all ob, so swap applies) to place the donor copies at the positions where constants will go.
(3) Apply $\mathsf{erase} : \mathsf{ob} \to \mathsf{unit}$ at constant positions to convert ob wires to unit wires.
(4) Apply the raw term encoding, which expects unit wires at constant leaf positions.

3.7 Dimension 3: rewrite rules and structural equations

3.7.1 Rewrite rules. Each TRS rule $\ell \to r$ becomes a 3-cell:

$$\mathsf{rule}_i : [\![\ell]\!] \to [\![r]\!]$$

where $[\![t]\!]$ denotes the 2-morphism encoding of t. Both sides have the same source (ob^k) and target (ob), so the 3-cell is well-formed.

3.7.2 Naturality of copy. For the structural cells to interact correctly with function symbols, we need *naturality* equations: the 3.7 cell commutes with function application. For a unary f:

$$f \cdot 3.7 = (f \otimes f)$$

For a binary f:

$$f \cdot 3.7 = (3.7) \cdot (\mathsf{ob}\ \mathsf{swap}\ \mathsf{ob}) \cdot (f \otimes f)$$

The right-hand side of the binary case uses a permutation to route the two copies of the first argument together and the two copies of the second argument together.

These are declared as *Eq attachments* in alifib: the type `Eq` is a small complex with cells `pt`, `dom`, `cod`, `lhs`, `rhs`, `dir`, `inv`, and attaching along it produces a pair of invertible 3-cells (one in each direction) witnessing the equation.

3.7.3 Structural equations. We also declare a handful of coherence cells:

$\mathrm{id}_1 \cdot \mathrm{id}_1 \leftrightarrow \mathrm{id}_1$	(identity is idempotent)
$\mathsf{swap} \cdot \mathsf{swap} \leftrightarrow \mathrm{id}_2$	(swap is an involution)
$\mathsf{swap} \leftrightarrow$	(copy is commutative)

Each is given as a pair of inverse 3-cells.

3.8 Summary of the encoding

Given a TRS (Σ, R), the output directed complex has:

Dim	Generators	Source
0	pt	—
1	ob, (optionally unit)	sort(s)
2	f for each $f \in \Sigma$	function symbols
	3.8, swap, (erase, unit_l, unit_r)	structural cells
	$\mathrm{id}_1, \mathrm{id}_2, \ldots$	identity cells
3	rule_i for each rule in R	rewrite rules
	Copy_f.dir/inv for each f	naturality
	involutions for swap, id_1, 3.8	coherence

The resulting complex is a finite presentation of the free $(\infty, 2)$-category generated by the TRS: the 2-cells present the term algebra, and the 3-cells present the rewriting relation. The higher cells (naturality, coherence) ensure that the structural operations interact with function symbols in the expected way.

Paths of 3-cells in this complex correspond exactly to rewrite sequences in the original TRS, modulo the structural rearrangements needed to match variables in consecutive rule applications.

References

[1] A. Burroni. Higher-dimensional word problems with applications to equational logic. *Theoretical Computer Science*, 115(1):43–62, 1993.

[2] Y. Guiraud and P. Malbos. Polygraphs of finite derivation type. *Mathematical Structures in Computer Science*, 29(8):1130–1192, 2019.